Complete Project Documentation: OmniSense-AI

Author: Mehak Mouazam (AI Intern)
Official GitHub Repository: https://github.com/Mehak-Maan/OmniSense-AI
Architecture: Multimodal Video RAG Engine | Temporal Video, Audio ASR, PDF & Vision Reasoning

---

Welcome to the official documentation for OmniSense-AI!
This document is written in clear, simple, and straightforward English to explain what this project is, how the multimodal technology works under the hood, how the end-to-end data pipeline functions, and how you can effortlessly set up and run the entire system on your local computer.

1. What is this Project?

OmniSense-AI is an advanced, enterprise-grade **Multimodal Video RAG** (Retrieval-Augmented Generation) assistant. While standard AI chatbots can only understand basic text messages, OmniSense-AI can analyze, cross-reference, and reason across **four distinct media modalities simultaneously**: Video, Audio, PDF Documents, and Images.

Understanding RAG (Retrieval-Augmented Generation) in Simple Terms:
Standard AI models often make up fake facts (known as *hallucinations*) when answering specific questions. RAG solves this by working in two distinct steps:
1. **Retrieval:** When you ask a question, the system searches your uploaded files to find the exact, relevant paragraphs, spoken sentences, or video scenes.
2. **Generation:** It feeds those exact facts into the AI brain (LLM), which produces a grounded, truthful, and verified response.

What can you do with OmniSense-AI?

- **Synchronized Video Scrubbing:** Upload an MP4 video, and the system extracts high-quality scene keyframes with visual descriptions.
- **Click-to-Jump Video Bookmarks:** When the AI mentions a moment in the video (such as 00:35), it provides an interactive timestamp button. Clicking it instantly seeks the video player to that exact second.
- **Speech Transcription (ASR):** Spoken audio tracks are converted into text transcripts with word-level millisecond precision.
- **Deep PDF Grounding:** Long annual reports, research papers, or manuals are parsed page-by-page with exact page citations.
- **Visual Diagram & Chart Q&A;:** Ingest system architecture flowcharts and financial graphs; AI vision reads all metrics and cards.
- **Dynamic Attention Distribution:** Live progress meters show how much of the answer originated from Video vs. PDF vs. Image.

2. What Technologies & Tools Were Used?

OmniSense-AI is built using a modern full-stack architecture combining high-performance Python backend services and an interactive React frontend:

Component	Technology / Tool	Role & Purpose in the System
Backend Engine	`FastAPI (Python 3.12)`	High-throughput asynchronous REST server handling multipart uploads, video extraction, and streaming chat requests.
Frontend Web UI	`React 19 + TypeScript + Vite`	Interactive user interface featuring a synchronized HTML5 video player, scene timeline cards, and real-time chat.
Styling & UX	`Tailwind CSS v4 + Lucide`	Modern glassmorphic aesthetic with dark and light theme toggles, dynamic badges, and responsive layouts.
Computer Vision	`OpenCV (Headless)`	Inspects video frame rates, resolution, duration, and extracts 320x180 thumbnail keyframes at strategic scene checkpoints.
Visual AI Vision	`Groq Vision (qwen3.8-27b)`	Converts architecture diagrams, charts, and keyframe snapshots into structured markdown tables and KPI metric blocks.
Audio Speech Recognition	`Groq Whisper Turbo`	Transcribes human speech from audio files or extracted video audio tracks with segment-level timestamp synchronization.
Document Intelligence	`PyMuPDF (fitz)`	Reads complex multi-page PDF documents and partitions them into semantic 300-word paragraph chunks with page tagging.
Unified Vector Index	`Scikit-Learn (TF-IDF)`	Combines text from all four modalities into a unified sparse vector space for sub-millisecond (<5ms) cosine similarity matching.
LLM Reasoning Brain	`Groq LLM (gpt-oss-20b/120b)`	Synthesizes accurate, natural-language answers anchored strictly to retrieved evidence without hallucinating fake facts.

3. How Does the Pipeline Work? (Step-by-Step Data Flow)

The OmniSense-AI architecture processes incoming data through four specialized extraction pipelines that converge into a unified vector retrieval space:

Scenario A: Video Ingestion & Temporal Scene Processing

1. **File Inspection:** OpenCV opens the uploaded MP4/WebM video and reads total frames, FPS, dimensions (e.g., 1920x1080), and length in seconds.
2. **Checkpoint Keyframing:** Keyframes are sampled at dynamic percentage checkpoints (such as 10%, 35%, 65%, and 90% timestamps).
3. **Visual Summarization:** Each keyframe thumbnail (320x180) is encoded and analyzed by Groq Vision to produce a 2-4 word scene title and description.
4. **Audio Track Extraction:** FFmpeg strips the 16kHz PCM WAV audio track and passes it directly to Groq Whisper for spoken speech transcription.

5. **Temporal Indexing:** Each scene is stored in the vector index along with its exact timestamp tag (e.g., 00:35).

Scenario B: PDF Document Processing

1. **Text Extraction:** PyMuPDF parses the document page-by-page.
2. **Paragraph Chunking:** Text is split into ~300-word logical chunks along natural paragraph boundaries.
3. **Provenance Tagging:** Every chunk retains its originating file name and exact page number (e.g., Page 1, Page 2).
4. **Vector Storing:** These chunks are added to the RAG memory for instant page-level citations.

Scenario C: Image & Diagram Analysis

1. **Base64 Encoding:** Image files are converted to high-resolution base64 data.
2. **Vision OCR & Layout Extraction:** Groq Vision extracts title headers, 4 KPI cards, flowcharts, tables, and system metrics verbatim.
3. **Section Breakdown:** The description is partitioned by markdown section headers (##) so specific sub-elements can be searched independently.

Scenario D: User Question Answering & Synthesis (The RAG Core)

1. **Query Transformation:** The user's prompt is vectorized using the fitted TF-IDF model.
2. **Cosine Similarity Matching:** The vector index ranks all stored chunks across all four modalities, returning the Top-6 highest-scoring matches.
3. **Dynamic Attention Weights:** Similarity scores are aggregated by modality to calculate active attention percentages (e.g., Video: 56%, PDF: 31%, Image: 13%).
4. **Timestamp Extraction:** Video and audio matches yield click-to-jump timestamp tags (e.g., 00:35, 01:30).
5. **Grounded LLM Response:** Groq LLM evaluates the verified context and produces a detailed response with exact citations and interactive playback buttons.

4. Project Folder Structure Explained

The codebase is cleanly organized into modular backend, frontend, and asset directories:

File / Folder	Description & Functionality
`backend/main.py`	The core FastAPI backend service. Houses OpenCV keyframe extraction, PyMuPDF parsing, Whisper ASR, TF-IDF vector retrieval, and Groq LLM synthesis.
`backend/requirements.txt`	Python dependencies: fastapi, uvicorn, opencv-python-headless, pymupdf, groq, scikit-learn, pillow, reportlab.
`backend/.env.example`	Configuration template for API keys (e.g., GROQ_API_KEY). Real keys are kept private via .gitignore.
`frontend/src/App.tsx`	The primary React application component. Features the HTML5 video player, dynamic keyframe timeline, attention weight meters, and real-time chat interface.
`frontend/package.json`	Frontend Node dependencies: React 19, TypeScript, Vite, Tailwind CSS v4, Lucide-react icons, React-Markdown.

`sample_videos/`	Pre-bundled sample MP4 tutorial video for instantaneous video RAG testing.
`sample_pdfs/`	Sample annual earnings PDF report containing financial metrics and tables for document Q&A; testing.
`sample_audios/`	Sample executive product strategy audio meeting recording for speech recognition testing.
`sample_images/`	Sample system architecture dashboard image containing KPI cards, data flowcharts, and metrics.
`README.md`	Complete GitHub repository documentation including architecture diagrams, API specs, and setup instructions.

5. Easy Setup Guide: How to Run on Your Computer

Follow these simple steps to launch the entire full-stack system locally:

```
# Step 1: Clone the Official GitHub Repository
git clone https://github.com/Mehak-Maan/OmniSense-AI.git
cd OmniSense-AI

# Step 2: Set Up and Start the Python Backend
cd backend
python -m venv venv
.\venv\Scripts\activate # (Linux/macOS: source venv/bin/activate)
pip install -r requirements.txt
# Create .env and insert your GROQ_API_KEY=gsk_...
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
Backend live at: http://localhost:8000 (API Docs: http://localhost:8000/docs)

# Step 3: Set Up and Start the React Frontend (In a New Terminal)
cd frontend
npm install
npm run dev
Frontend live at: http://localhost:5173

# Step 4: Open Browser and Explore
Navigate to http://localhost:5173 in your web browser. Click 'Load Demo Video & Data' in the
top navigation bar to test synchronized video scrubbing, document search, and chat reasoning
instantly.
```

Author Information & Official Project Links:

- **Project Author:** Mehak Mouazam (AI Intern)
- **Official GitHub Repository:** https://github.com/Mehak-Maan/OmniSense-AI
- **Core Highlights:** 4-Modality Unified RAG, Sub-second Groq Inference, Synchronized Timeline, and Dynamic Attention Weights.
- **License:** MIT Open Source License. Contributions and feedback are welcome!